

ID	StimulusGeneration	OutputChecking	Coverage
R1	Write various data patterns to all R/W registers (CTRL at 0x00 CLK_DIV at 0x10 SS_CTRL at 0x14 INT_EN at 0x18 DELAY at 0x20) via apb_write transactions.	Perform consecutive apb_read operations to the same register addresses immediately following the writes to verify that the register retains the pattern.	Read/write access to all register bits across all valid R/W registers.
R2	Execute an apb_reset_seq to apply a reset pulse via PRESETn followed by reading every register address from 0x00 to 0x20.	Verify that the returned data matches the expected reset defaults: CTRL=0x00 STATUS=0x00000012 CLK_DIV=0 SS_CTRL=0 INT_EN=0 INT_STAT=0 DELAY=0.	Read access to every register address immediately post-reset.
R3	Write 32'h00000000 to CTRL to clear CTRL.EN. Attempt a TX FIFO write to 0x08. Assert SS_CTRL configurations at 0x14. Toggle EN 0->1->0 with data in the FIFO. Write CPOL=0 and CPOL=1 with EN=0.	Verify that the TX write is ignored and STATUS.TX_EMPTY remains 1. Verify that SS_n lines stay high and STATUS.BUSY remains 0. Verify that the FIFO flushes on EN 1->0 transition and SCLK stays at its CPOL idle state.	CTRL.EN toggling states and transactions attempted during the disabled state.
R4	Call do_transfer with all 4 combinations of SPI modes (Mode 0: 2'b00 Mode 1: 2'b01 Mode 2: 2'b10 Mode 3: 2'b11) via the ctrl_val setting.	Verify via protocol monitoring or loopback reads that SCLK idle polarity perfectly matches the CPOL configuration for each selected mode.	All 4 SPI operational modes (Modes 0 1 2 and 3).
R5	Transmit 8-bit 16-bit and 32-bit words using loopback mode with varying SPI clock phases and polarities.	Verify that the MOSI line changes state strictly at the configured launch edge and remains perfectly stable at the sample edge.	MOSI signal timing stability and data integrity across all 4 clock modes.
R6	Initiate transfers configuring ctrl_val with LSB_FIRST=0 (MSB-first) and LSB_FIRST=1 (LSB-first) across 8-bit 16-bit and 32-bit configurations.	Verify via loopback verification that received data bit order matches the transmitted bit stream ordering precisely.	Combinations of data width (8/16/32-bit) crossed with bit ordering configurations.
R7	Transmit data blocks of 8-bit 16-bit and 32-bit widths via apb_write to 0x08 while asserting SS_EN[0].	Verify that STATUS.BUSY asserts exactly for the duration of the transfer width (8 16 or 32 SCLK cycles) and drops immediately upon completion.	Transfer width durations (8 16 and 32 clock cycles) and STATUS.BUSY tracking.
R8	Execute do_transfer loops across clock divider values (DIV=0 DIV=1 DIV=3 DIV=255).	Verify that data pops out in the identical First-In First-Out sequence (0xA1 through 0xA8) as it was captured.	CLK_DIV range coverage including low mid and high corner values.
R9	Write 8 consecutive data words to TX_DATA (0x08) while keeping the SPI clock slow (DIV=99) to keep the FIFO full.	Verify that all 8 words are accepted in sequence by checking STATUS updates after each push transaction.	Sequential write operations to the TX FIFO under full/near-full conditions.
R10	Fill the RX FIFO using loopback mode transfers and perform 8 consecutive apb_read transactions from RX_DATA (0x0C).	Verify that data pops out in the identical First-In First-Out sequence (0xA1 through 0xA8) as it was captured.	Sequential read operations popping the RX FIFO from full to empty.
R11	Push 8 distinct words into the TX FIFO sequentially without letting a transfer drain them.	Verify that STATUS.TX_FULL asserts on the 8th write and no earlier indicating a depth of exactly 8 entries.	TX FIFO depth boundary testing (exactly 8 entries).
R12	Generate 8 loopback transfers sequentially with SS asserted to completely fill the receiving queue.	Verify that STATUS.RX_FULL asserts exactly upon the completion of the 8th received entry.	RX FIFO depth boundary testing (exactly 8 entries).
R13	Perform a 9th write operation to TX_DATA (0x08) while STATUS.TX_FULL is fully asserted.	Verify that the 9th write data is completely discarded and that STATUS.TX_OVF and INT_STAT.TX_OVF are set.	TX FIFO overflow generation and status/interrupt flag tracking.
R14	Trigger an additional SPI transfer via loopback while the RX FIFO is already filled with 8 entries.	Verify that the incoming 9th word is discarded and that STATUS.RX_OVF and INT_STAT.RX_OVF flags assert.	RX FIFO overflow generation and status/interrupt flag tracking.
R15	Perform an apb_read from RX_DATA (0x0C) while the block is disabled or when STATUS.RX_EMPTY is 1.	Verify that the read transaction returns a value of 0 and that the RX overflow flags are NOT erroneously set.	RX FIFO underflow read handling on an empty status.
R16	Write 0 to INT_EN to mask interrupts and trigger a TX_OVF event. Then unmask INT_EN[0] (TX_EMPTY) and run a transfer to empty.	Verify that INT_STAT captures the masked event (TX_OVF=1) without asserting the external IRQ line. Verify that the unmasked event successfully drives the external IRQ signal.	All 5 interrupt status bits being captured when masked and driving IRQ when unmasked.
R17	Write 32'h00000000 to INT_STAT to observe no change. Then write 32'h00000001 to clear bit 0 followed by 32'h0000001F to clear all.	Verify that writing 0 has no effect on the status flags and that writing a 1 to a specific bit successfully clears it (W1C behavior).	W1C functional clearing logic for all 5 interrupt status flags.
R18	Fill the TX FIFO to 8 entries write a 9th word to trigger an overflow event and simultaneously issue a W1C clear write to the TX_OVF bit.	Verify that due to the race condition the interrupt status bit remains high (1) because the hardware event takes priority over the clear.	Simultaneous hardware interrupt event and software clear operation (W1C)
R19	Configure CTRL with LOOPBACK=1 and execute 8-bit 16-bit and 32-bit transfers without external MISO stimulation.	Verify that the internal MOSI is routed directly to the RX shifter and that the received words match the transmitted words exactly.	Loopback mode functionality across all supported data widths (8 16 and 32 bits)
R20	Write different combinations to SS_CTRL (0x14) varying both SS_EN and SS_VAL across all 4 available lanes.	Verify that the output lines follow the combinational equation SS_n[i] = !SS_EN[i] SS_VAL[i] instantaneously without needing an active clock.	Combinational state mapping for all 4 SS_n output lines across all EN_n
R21	Configure DELAY to 0 1 and 128 in the DELAY register (0x20) and issue multiple consecutive data writes to the TX FIFO.	Verify that the hardware inserts exactly the configured number of SCLK half-cycles as an idle delay between the consecutive transfers.	Inter-transfer delay insertion tracking for zero small and large delay settings
R22	Execute a wide range of read and write transactions to various valid register offsets across all test sequences.	Verify that the APB bus signals PSLVERR remains 0 and PREADY remains 1 for every single access demonstrating zero wait states.	Bus error and ready tracking across all valid interface interactions.
R23	Attempt read and write operations to undefined or reserved register address offsets such as 0x24 0x28 0x30 and 0xFF.	Verify that all reads from these addresses return 32'h00000000 and that write data patterns are entirely ignored with no impact on functionality.	Address decoding boundaries and robustness testing against out-of-range
R24	Configure CLK_DIV to 16'h0000 and initiate a standard SPI transfer sequence.	Verify that the resulting SCLK frequency operates at its maximum velocity of exactly PCLK / 2.	Clock divider edge case verification at the absolute minimum divider value
R25	Initiate a slow transfer (DIV=3) and immediately write a new value (DIV=0xFF) to the CLK_DIV register mid-flight.	Verify that the ongoing transfer completes using the original parameters proving configuration fields are sampled strictly at start.	Mid-transfer register update isolation and configuration stability parameters